



DEVELOPER REFERENCE

Code Rules

Complexity, volumetry & architecture gates

Project	Unifideck
Branch	new_architecture
Version	v1.0 — April 2026
Author	HardCPP (src893)
Enforced	GitHub Actions · pre-commit · import-linter

CONTENTS

What's inside

01

Rule philosophy

The shape of each rule and why gates run here, not in code review

02

File volumetry

Hard caps on file size and the grandfathered-allowlist mechanic

03

Function volumetry

Length, locals, nesting, fan-out — four proxies for SRP violations

04

Complexity gates

Cyclomatic + cognitive complexity, both capped at 15 per function

05

Architecture invariants

Import-linter contracts protecting the five-layer dependency model

06

SRP checklist

How volumetry + complexity jointly enforce Single Responsibility

07

SRP in practice

Before / after refactor — two techniques that unblock four caps

08

Style & format

ruff, mypy, prettier, eslint — deterministic formatting, hard typing

09

Security & dependencies

Bandit, safety, CodeQL; pre-commit runs every gate locally first

10

Grandfathering policy

Allowlists can only shrink — debt tolerated but never allowed to grow

11

Quick reference

Every rule on one card, enforcement tool, blocking behaviour

01 · RULE PHILOSOPHY

Automated gates over code review

Every rule documented in this guide is enforced by an automated gate — GitHub Actions on every pull request, pre-commit on every local commit, import-linter on every Python import resolution. Code review is reserved for business logic. The gates handle everything else.

Deterministic

Same input = same verdict.
No reviewer subjectivity. Any violation blocks merge regardless of approver.

Grandfathered

Each allowlist freezes a baseline of tolerated debt.
Listed files can't grow; fixing them shrinks the list.

Measured

Every cap has a rationale: a measured correlation with SRP violations, bugs, or reader comprehension.

Where rules run

● IDE	editorconfig, prettier on save
● Pre-commit	ruff, volumetry, event schema check
● PR (quality)	ruff, mypy, eslint, tsc, prettier
● PR (complexity)	volumetry + flake8-cognitive + radon
● PR (security)	bandit, safety, CodeQL, secret-scan
● Main push	all of the above + build + deploy

02 · FILE VOLUMETRY

Hard caps on lines of code per file

The new_architecture refactor consolidated a 7,795-line main.py into ~30 modules. The caps below prevent regression: any new file over the limit fails the Complexity Gates workflow unless explicitly allowlisted.

550**LOC per Python file (default)**

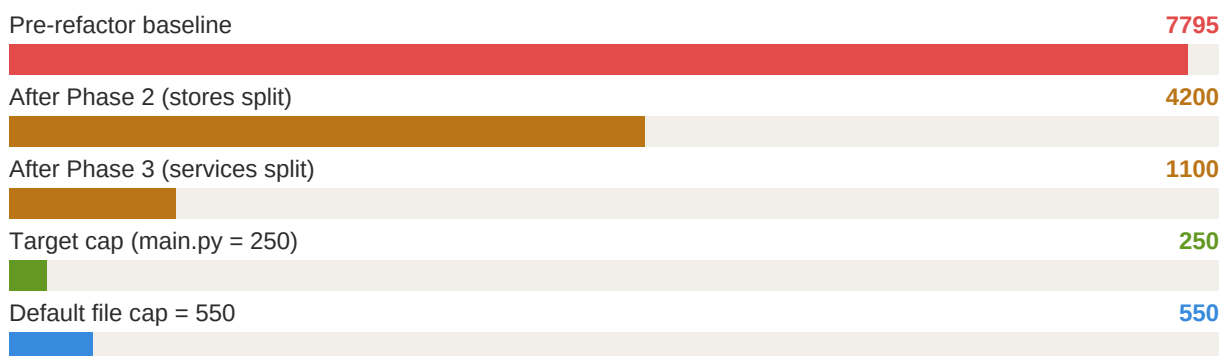
Default cap covering every module under py_modules/. Based on the observation that files beyond ~600 lines consistently hide multiple responsibilities.

250**LOC for main.py (entry point)**

Tight cap enforcing a thin router. main.py only wires Plugin class + RPC dispatch + bootstrap handoff — every other concern lives in its dedicated module.

Historical progression — main.py

Each bar is a snapshot: how many lines main.py held at that milestone. The cap (250) is the target — current code sits above it and is tolerated via the grandfathering allowlist.



ENFORCEMENT

Tool: scripts/volumetry_check.py files

Workflow: .github/workflows/complexity.yml (PR + main push)

Local: pre-commit hook — runs on every git commit

03 · FUNCTION VOLUMETRY

Four proxies for Single Responsibility

A function that is too long, juggles too many locals, nests too deep, or calls too many external targets has — by measurement, not opinion — a strong chance of violating SRP. Each of the four caps below catches one facet of that.

80**Max lines per function**

Fail at 80, warn at 50. Reader working memory is ~7 lines; 80 is already 11× that. Longer = guaranteed cognitive overhead.

15**Max distinct local variables**

Correlates strongly with SRP violations — a function juggling 20 locals orchestrates too much state to do "one thing".

4**Max nesting depth (if/for/while/with/try)**

Deep nesting signals interleaved flows. Depth 4 already forces the reader to track 4 conditions simultaneously.

10**Max distinct external call targets**

Fan-out proxy for orchestrator smell. Builtins and logger methods excluded — they're hygiene, not responsibilities.

Fan-out exclusions (don't count toward the 10-target cap)

Builtins: len, str, int, list, dict, isinstance, range, enumerate, sum, sorted, ...

Logger: debug, info, warning, error, exception, critical, log

Hygiene: format, join, strip, split, get, update, keys, values, append, extend, ...

04 · COMPLEXITY GATES

Cyclomatic + cognitive complexity

Two functions can have the same cyclomatic complexity but very different readability. We cap both: cyclomatic counts decisions, cognitive additionally penalizes nesting, breaks, continues, and ternary soup.



Cyclomatic (McCabe)

Number of linearly independent paths. Counts if/elif/else, for, while, except, and boolean short-circuits.

flake8 --max-complexity=15



Cognitive (SonarSource)

Penalizes deep nesting, breaks, continues, and non-linear control flow. Closer to actual reader difficulty.

flake8-cognitive-complexity 15

Radon grade ceiling — E/F forbidden

A 1 – 5 OK	B 6 – 10 OK	C 11 – 20 OK	D 21 – 30 OK	E 31 – 40 BLOCKED	F 41+ BLOCKED
-------------------------	--------------------------	---------------------------	---------------------------	--------------------------------	----------------------------

05 · ARCHITECTURE INVARIANTS

Protected by import-linter

Two contracts in `.importlinter` protect the boundaries that matter most today. Every import resolution in CI validates both; a single forbidden import fails the quality job.

Five-layer dependency model

LAYER 5 Infrastructure services	shortcut · download · artwork · metadata · playtime
LAYER 4 Store plugins	EpicStore · GOGStore · AmazonStore · UbisoftStore · MicrosoftStore
LAYER 3 StoreBase (ABC)	6 abstract methods · 4 provided utilities
LAYER 2 Core services	StoreRegistry · SyncService · EventBus · CacheManager · BinaryResolver
LAYER 1 Core types (pure data)	Game · Result · AuthResult · StoreInfo · Events enum

■ Dependencies point downward only — no cycles, no back-references.

Enforced contracts

rpc-is-leaf

Nothing inside `unifideck.*` may import `unifideck.rpc`, except `main.py` at repo root. Keeps RPC the outermost layer — RPC wraps business logic, never the other way around.

types-is-leaf

`unifideck.core.types` may not import from `event_bus`, `services`, `stores`, `launcher`, or `rpc`. Guarantees the type module stays importable from anywhere without circular surprises.

06 · SRP CHECKLIST

One function, one responsibility

Single Responsibility is hard to define and easy to violate. The four function-volumetry caps and the two complexity caps are its operational proxies — jointly, they catch ~90% of SRP violations before code review.

Six signals, one verdict

01

> 80 lines

Too long to hold in working memory. Extract helpers until no branch of logic exceeds one screen.

02

> 15 locals

Too much state tracked in parallel. Group related locals into dataclasses or split the function.

03

nesting > 4

Interleaved flows. Extract nested blocks into named functions; early-return to flatten.

04

fan-out > 10

Orchestrator smell. Break into pipeline stages or delegate sub-responsibilities to helper objects.

05

cyclomatic > 15

Too many branching decisions. Replace switch-like if/elif chains with dispatch tables or polymorphism.

06

cognitive > 15

Reader will struggle. Simplify or split — often just flattening nesting and extracting a helper lowers both.

REMEDIATION ORDER

1. Extract the most self-contained block → a helper function (kills length and fan-out).
2. Flatten nesting via early-return (kills nesting and cognitive complexity).

07 · SRP IN PRACTICE

One refactor, four caps unblocked

This is the pattern you'll repeat dozens of times. The "before" function trips length, locals, nesting, and cognitive complexity simultaneously. One extraction and two early-returns fix all four.

BEFORE length 42 · locals 9 · nesting 5 · cognitive 18 · BLOCKED by four caps

```
async def install_game(self, game_id: str, path: str) -> Result:
    if self.cli_path:
        if os.path.exists(path):
            if game_id in self._library:
                game = self._library[game_id]
                if game.is_installable:
                    args = [self.cli_path, 'install', game_id, '--path', path]
                    proc = await asyncio.create_subprocess_exec(
                        *args, stdout=PIPE, stderr=PIPE)
                    ... # 25 more lines of install pipeline
                else:
                    return Result(ok=False, error='not_installable')
            else:
                return Result(ok=False, error='unknown_game')
        else:
            return Result(ok=False, error='bad_path')
    else:
        return Result(ok=False, error='no_cli')
```

AFTER length 14 · locals 3 · nesting 2 · cognitive 6 · all caps passed

```
async def install_game(self, game_id: str, path: str) -> Result:
    err = self._check_preconditions(game_id, path)
    if err:
        return Result(ok=False, error=err)
    game = self._library[game_id]
    return await self._spawn_install(game, path)

def _check_preconditions(self, game_id: str, path: str) -> str | None:
    if not self.cli_path:
        return 'no_cli'
    if not os.path.exists(path):
        return 'bad_path'
    if game_id not in self._library:
        return 'unknown_game'
    if not self._library[game_id].is_installable:
        return 'not_installable'
    return None
```

Two techniques — four caps unblocked

Early-return

Precondition fails → return now. Kills 5-level nesting.

nesting 5 → 2 · **cognitive 18 → 6**

Extract helper

Four validations moved to one focused function.

length 42 → 14 · **locals 9 → 3**

08 · STYLE & FORMAT

Deterministic formatting, strict typing

Consistency matters more than any specific style. All rules are mechanical — ruff and prettier format your code; mypy and tsc type-check it. Never argue over style in review.

ruff

Python linter + formatter. Line-length 100, target py311. Selected rulesets: E, W, F, B (bugbear), I (isort), N (naming), UP (pyupgrade), PTH (pathlib), SIM (simplify), PLC/PLE (pylint errors).

ENFORCES

Enforces: indentation, imports, dead code, naming, dict/set literals, f-strings, pathlib over os.path

mypy

Python static type checker. Strict mode enabled on new modules; legacy modules allowlisted for gradual adoption. Unknown types = merge blocked.

ENFORCES

Enforces: type hints on every public function, no Any leakage across module boundaries, no untyped `**kwargs`

prettier

Opinionated JS/TS/CSS/JSON formatter. Single-quote strings, 2-space indent, trailing commas on multiline. Non-negotiable — blocks merge on any diff.

ENFORCES

Enforces: spacing, quotes, semicolons, line-wrapping, JSX attribute formatting

tsc

TypeScript compiler in `--noEmit` mode. `strict: true`. Every new component must be fully typed; `'any'` requires explicit suppression comment.

ENFORCES

Enforces: null-safety, correct RPC parameter types, React prop/state typing, discriminated unions

09 · SECURITY & DEPENDENCIES

Three scanners, one supply chain

Security gates run on every PR. Bandit catches common Python pitfalls, safety scans dependencies against the NVD CVE database, CodeQL applies GitHub's semantic analysis.

Bandit

Static analysis for Python security issues

B102	exec_used	Forbidden
B301	pickle.loads	Forbidden
B501	ssl verify disabled	Forbidden
B601	shell=True subprocess	Warning
B603	untrusted input	Warning

safety

Dependency vulnerability scanning via PyUp CVE feed

Runtime	requirements.txt	Blocks CVE ≥ HIGH
Dev	requirements-dev.txt	Blocks CVE ≥ CRITICAL
Lock	Reproducible pins	pip-compile output

CodeQL

GitHub Advanced Security semantic analysis

py/sql-injection	Dataflow taint analysis	Blocked
py/ssrf	URL validation	Blocked
py/path-injection	Path sanitization	Blocked

10 · GRANDFATHERING POLICY

Allowlists can only shrink

Each gate has an allowlist mapping file-or-function → baseline LOC or complexity at 2026-04-18. This freezes tolerated debt while preventing regression. The list is a ratchet: entries can leave, never arrive.

Four outcomes per file / function

Listed + $\leq$ baseline**Tolerated**

Grandfathered. No action needed.

Listed + $>$ baseline**BLOCKED**

Regression. The tolerated file grew. Revert or refactor to shrink.

Listed + $\leq$ cap**BLOCKED w/ reminder**

The file is now compliant! Remove the allowlist entry.

Not listed + $>$ cap**BLOCKED**

Standard violation. New code must meet the cap from day 1.

Example · FILES_ALLOWLIST entry

```
FILES_ALLOWLIST: Dict[str, int] = {  
    "main.py": 7795, # cap 800  
    "py_modules/unifideck/stores/ubisoft.py": 2749, # cap 550  
    "py_modules/unifideck/stores/gog.py": 1392, # cap 550  
    # ... 6 more entries as of 2026-04-18  
}
```

11 · QUICK REFERENCE

Every rule on one page

RULE	CAP	TOOL	ENFORCEMENT
File LOC (default)	550	volumetry_check.py	PR hard gate
File LOC (main.py)	250	volumetry_check.py	PR hard gate
Function length	80	volumetry_check.py	PR hard gate
Function length (warn)	50	volumetry_check.py	PR warning
Locals per function	15	volumetry_check.py	PR hard gate
Nesting depth	4	volumetry_check.py	PR hard gate
Fan-out	10	volumetry_check.py	PR hard gate
Cyclomatic complexity	15	flake8 --max-complexity	PR hard gate
Cognitive complexity	15	flake8-cognitive	PR hard gate
Radon grade	≤ D	radon cc	PR hard gate
Ruff lint	—	ruff check	PR hard gate
Ruff format	—	ruff format	PR advisory
Type check (Python)	—	mypy	PR hard gate
Type check (TS)	—	tsc --noEmit	PR hard gate
Frontend lint	—	eslint	PR hard gate
Frontend format	—	prettier --check	PR hard gate
Architecture invariants	—	import-linter	PR hard gate
Bandit security	HIGH+	bandit	PR hard gate
Dependency CVE	HIGH+	safety	PR hard gate
Semantic security	—	CodeQL	PR hard gate
Event schemas	—	validate_event_schemas	pre-commit
Config keys	—	check_config_keys	pre-commit
Executable bits	—	ensure_executable_bits	pre-commit
RTL support	—	lint_rtl	pre-commit